

Seaborn, mapped to your syllabus

Seaborn is a statistical plotting library built on Matplotlib. In this course it is not a separate topic — it is the tool you use *inside* every unit: to inspect features before scaling them, to see whether a regression line actually fits, to look at how classes separate in feature space, and to read a dendrogram after hierarchical clustering.

Scope of these notes. Rather than covering seaborn's entire API, each unit below covers the small set of functions that earns its place in *that* unit's workflow, with full definitions, every common calling pattern (the "cases"), worked examples using scikit-learn style datasets, and an exercise set. Install once, use everywhere:

```
pip install seaborn matplotlib pandas scikit-learn
```

```
# the import convention used throughout these notes
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="whitegrid") # sets a clean default look for every plot below
```

UNIT 2

Preprocessing — seeing your features before you touch them

Feature scaling, feature selection, dimensionality reduction (PCA). Before you scale a feature you should look at its distribution; before you select or drop a feature you should look at how correlated it is with everything else. Three functions do almost all of this work.

`sns.histplot()` DISTRIBUTION

• Definition

Draws a histogram (optionally with a KDE curve overlaid) of a numeric column, showing how its values are spread out. Figure-level histograms would need Matplotlib boilerplate; `histplot` is the axes-level, one-line version seaborn provides.

```
sns.histplot(data=None, x=None, y=None, hue=None, stat='count',
             bins='auto', kde=False, element='bars', multiple='layer', ax=None)
```

PARAMETER	WHAT IT CONTROLS
<code>data, x</code>	a DataFrame and the column name to plot
<code>hue</code>	splits the histogram by a categorical column (e.g. class label)
<code>stat</code>	'count', 'density', or 'probability' on the y-axis
<code>bins</code>	number of bins, or a strategy like 'auto'
<code>kde</code>	overlays a smooth density curve on top of the bars
<code>multiple</code>	when using <code>hue</code> : 'layer', 'stack', or 'dodge'

• Why it matters here

Feature scaling decisions depend on shape. A right-skewed feature (e.g. income) is often log-transformed before standardization; a roughly symmetric feature is a good candidate for z-score scaling as-is. You check the shape with `histplot` first.

• Every case

- **Single feature, raw:** `sns.histplot(data=df, x="Age")`
- **With a smooth curve:** `sns.histplot(data=df, x="Age", kde=True)`
- **Before vs after scaling, side by side** using two subplots
- **Split by class** using `hue="target"` — reveals whether a feature separates classes before you even train a model
- **Density instead of raw counts:** `stat="density"` — useful when comparing groups of different sizes

• Worked example

```
# Unit 2: check a feature's distribution before scaling
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

wine = load_wine(as_frame=True)
df = wine.frame

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.histplot(data=df, x="proline", kde=True, ax=axes[0], color="#4C72B0")
axes[0].set_title("proline - before scaling")

scaled = df.copy()
scaled["proline"] = StandardScaler().fit_transform(df[["proline"]])
sns.histplot(data=scaled, x="proline", kde=True, ax=axes[1], color="#DD8452")
axes[1].set_title("proline - after StandardScaler")
```

```
plt.tight_layout()
plt.show()
```

Notice the *shape* of the histogram is identical before and after — standardization re-centers and rescales, it does not change the distribution's shape. That's a common misconception worth confirming visually.

sns.boxplot() DISTRIBUTION / OUTLIERS

• Definition

Draws a box (interquartile range), a line at the median, whiskers extending to $1.5 \times \text{IQR}$, and individual points beyond that as outliers. It is the standard way to spot outliers that would distort a scaler like `MinMaxScaler`.

```
sns.boxplot(data=None, x=None, y=None, hue=None, orient=None,
            width=0.8, showfliers=True, ax=None)
```

PARAMETER	WHAT IT CONTROLS
<code>x, y</code>	categorical column on one axis, numeric on the other
<code>hue</code>	a second categorical grouping, drawn as side-by-side boxes
<code>showfliers</code>	toggle the outlier points off if you only want the box
<code>orient</code>	force 'h' or 'v' orientation

• Every case

- **Single numeric column:** `sns.boxplot(data=df, y="proline")`
- **Numeric feature by category:** `sns.boxplot(data=df, x="target", y="proline")` — compares a feature's spread across classes
- **Many features at once** after melting a wide DataFrame to long form, to scan every column for outliers in one figure
- **Grouped boxes:** add hue for a second categorical split

• Worked example — scanning all features for outliers

```
# reshape wide -> long so every feature becomes one row per value
long_df = df.drop(columns="target").melt(var_name="feature", value_name="value")

plt.figure(figsize=(11, 5))
sns.boxplot(data=long_df, x="feature", y="value")
plt.xticks(rotation=60, ha="right")
plt.title("Outlier scan across all raw features")
```

```
plt.tight_layout()
plt.show()
```

If a feature's scale dwarfs the others (e.g. `proline` in the hundreds vs. `hue` around 1), its box will flatten every other box to a line — that visual flattening is itself the argument for feature scaling.

`sns.heatmap()` CORRELATION / FEATURE SELECTION

• Definition

Renders a 2-D matrix (e.g. a correlation matrix, a confusion matrix, or PCA loadings) as a grid of colored cells, with color intensity encoding the value. It is the single most-used seaborn function across this entire syllabus because so much of ML reduces to a matrix.

```
sns.heatmap(data, vmin=None, vmax=None, cmap=None, annot=False, fmt='.2g',
            center=None, square=False, cbar=True, mask=None, ax=None)
```

PARAMETER	WHAT IT CONTROLS
<code>data</code>	a 2-D array or DataFrame (e.g. <code>df.corr()</code>)
<code>annot</code>	writes the numeric value inside each cell
<code>cmap</code>	color map, e.g. <code>'coolwarm'</code> for signed correlations
<code>center</code>	value mapped to the middle of the color scale (use <code>0</code> for correlation matrices so positive/negative are visually symmetric)
<code>mask</code>	boolean array to hide cells — commonly used to show only the lower triangle of a symmetric correlation matrix

• Every case

- **Full correlation matrix:** `sns.heatmap(df.corr(), annot=True, cmap="coolwarm", center=0)`
- **Lower-triangle only** (avoids repeating symmetric information) via a mask
- **PCA component loadings:** rows = components, columns = original features — reveals which original features drive each principal component
- **Confusion matrix** for classification (see Unit 4)

• Worked example — correlation matrix for feature selection

```
corr = df.drop(columns="target").corr()
mask = None
import numpy as np
mask = np.triu(np.ones_like(corr, dtype=bool)) # hide upper triangle
```

```
plt.figure(figsize=(9, 7))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
            center=0, square=True, cbar_kws={"shrink": .7})
plt.title("Feature correlation – candidates for removal are near  $\pm 1$ ")
plt.tight_layout()
plt.show()
```

• Worked example – PCA loadings

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(df.drop(columns="target"))
pca = PCA(n_components=4).fit(X)

loadings = pd.DataFrame(pca.components_,
                        columns=df.drop(columns="target").columns,
                        index=[f"PC{i+1}" for i in range(4)])

plt.figure(figsize=(11, 4))
sns.heatmap(loadings, cmap="vlag", center=0, annot=False)
plt.title("PCA component loadings")
plt.tight_layout()
plt.show()
```

EXERCISE – UNIT 2

Preprocessing with seaborn

1. Load the `load_breast_cancer` dataset from scikit-learn. Use `histplot` with `hue="target"` to find one feature that visibly separates the two classes and one that does not.
2. Use `boxplot` on the melted (long-form) feature set to identify the three features with the widest spread of outliers.
3. Compute the correlation matrix and use `heatmap` (masked, lower-triangle) to find a pair of features correlated above 0.9 – these are candidates to drop before modelling.
4. Run PCA with 2 components on the scaled data and heatmap the loadings to name the feature that dominates PC1.

► [Show a starting-point solution](#)

Regression — is the line actually a good fit?

Linear regression (one and many variables), gradient descent, overfitting, regularization, evaluation metrics. Seaborn won't run gradient descent for you, but it will show you, in one line, whether a fitted line under- or over-fits — which is exactly the visual judgment those topics ask you to make.

`sns.regplot()` AXES-LEVEL

• Definition

Plots a scatter of x vs y and fits + draws a regression line through it, with a shaded confidence interval. It is an *axes-level* function — it draws onto a single Matplotlib axis, so it composes with subplots.

```
sns.regplot(data=None, x=None, y=None, order=1, ci=95, scatter=True,
            fit_reg=True, robust=False, logistic=False, lowess=False, ax=None)
```

PARAMETER	WHAT IT CONTROLS
<code>order</code>	1 = linear; 2/3 fits a polynomial — set higher to visualize overfitting
<code>ci</code>	confidence interval band width (as a %), or <code>None</code> to hide it
<code>robust</code>	uses a robust regression, downweighting outliers
<code>logistic</code>	fits a logistic curve instead — useful preview for Unit 4
<code>lowess</code>	fits a local, non-parametric smoother instead of a straight line

• Every case

- **Simple linear regression:** `sns.regplot(data=df, x="RM", y="MEDV")`
- **Polynomial fit:** `order=2` or `order=3` — compare against `order=1` side-by-side to see under/overfitting
- **No scatter, line only:** `scatter=False` — clean when overlaying several fits
- **No confidence band:** `ci=None` — for a less cluttered look
- **Robust regression:** `robust=True` — downweights outliers so the line isn't dragged by them

• Worked example

```
# Unit 3: simple linear regression with one variable
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing(as_frame=True)
hdf = housing.frame.sample(1000, random_state=0) # subsample for a readable plot

plt.figure(figsize=(6, 5))
sns.regplot(data=hdf, x="MedInc", y="MedHouseVal",
            scatter_kws={"alpha": 0.3, "color": "#4C72B0"},
            line_kws={"color": "#C44E52"})
```

```
plt.title("Median income vs. median house value")
plt.show()
```

• Worked example — comparing polynomial orders (overfitting)

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
for ax, order in zip(axes, [1, 2, 6]):
    sns.regplot(data=hdf, x="MedInc", y="MedHouseVal", order=order,
                scatter_kws={"alpha": 0.2}, ax=ax, ci=None)
    ax.set_title(f"order = {order}")
plt.tight_layout()
plt.show()
```

At order=6 the curve starts chasing individual points near the edges of the data — the visual definition of overfitting, before you've computed a single metric.

sns.lmplot() FIGURE-LEVEL

• Definition

The figure-level counterpart to `regplot`. It combines a regression fit with faceting: it can draw one regression line per category, or lay out a whole grid of small regression plots split by one or two categorical variables. Use this specifically when you have **multiple variables** to compare, not just one.

```
sns.lmplot(data, x, y, hue=None, col=None, row=None, order=1,
           col_wrap=None, height=4, aspect=1, ci=95)
```

PARAMETER	WHAT IT CONTROLS
<code>hue</code>	separate colored regression lines in the <i>same</i> axes
<code>col, row</code>	separate <i>panels</i> (facets), one per category
<code>col_wrap</code>	wrap facets onto multiple rows after N columns
<code>height, aspect</code>	size of each individual facet

• Every case

- **One fit per category, same axes:** `hue="ocean_proximity"`
- **One fit per category, separate panels:** `col="ocean_proximity"`
- **Two-way facet grid:** both `row` and `col` for two categorical variables at once
- **Multiple linear regression, visualized one predictor at a time:** loop the target against each numeric predictor to sanity-check individual relationships even though the model uses all of them together

• Worked example — multiple variables, faceted

```
# regression of house value on income, split by coastal proximity
sample = housing.frame.sample(2000, random_state=1)
sample["ocean_group"] = pd.qcut(sample["Latitude"], 3, labels=["south", "mid", "north"])

sns.lmplot(data=sample, x="MedInc", y="MedHouseVal", col="ocean_group",
           scatter_kws={"alpha": 0.2}, height=4, ci=None)
plt.show()
```

sns.residplot() DIAGNOSTICS

• Definition

Fits a regression of y on x and plots the *residuals* (actual – predicted) instead of the raw fit. A good linear fit shows residuals scattered randomly around zero; a curved or funnel-shaped residual pattern signals that the linear model is mis-specified or that regularization / a different feature transform is needed.

```
sns.residplot(data=None, x=None, y=None, lowess=False, order=1, ax=None)
```

• Every case

- **Basic residual check:** `sns.residplot(data=df, x="RM", y="MEDV")`
- **With a smoother over the residuals:** `lowess=True` — makes a subtle curved pattern easier to spot
- **After regularization:** pass in the model's own residuals (`y_true - y_pred`) as a manual scatterplot to compare Ridge vs Lasso vs plain OLS

• Worked example

```
plt.figure(figsize=(6, 4))
sns.residplot(data=hdf, x="MedInc", y="MedHouseVal", lowess=True,
              line_kws={"color": "#C44E52"})
plt.axhline(0, color="grey", linestyle="--")
plt.title("Residuals: median income model")
plt.show()
```

A residual plot that fans out (variance grows with x) is *heteroscedasticity* — a linear model's constant-variance assumption is violated, and this is often the visual cue that motivates trying Ridge/Lasso or a transformed feature.

Regression with seaborn

1. Using the California housing data, plot `regplot` for AveRooms vs MedHouseVal and identify whether the relationship looks linear.
2. Use `lmpplot` with `col` to compare that same relationship split into three income quantile groups (`pd.qcut(df["MedInc"], 3)`). Does the slope change across groups?
3. Fit polynomial orders 1, 2, and 4 with `regplot` on AveOccup vs MedHouseVal (clip extreme values first) and describe which one looks overfit.
4. Use `residplot` to check whether a linear fit of MedInc vs MedHouseVal shows heteroscedasticity.

► [Show a starting-point solution](#)

UNIT 4

Classification — how do the classes sit in feature space?

Decision trees, Naive Bayes, logistic regression, k-NN, perceptron/MLP, neural networks, SVM, and classification metrics. Whatever the algorithm, the same three seaborn tools apply: look at the raw feature space with class colors, look at every pair of features at once, and, after training, look at the confusion matrix.

sns.scatterplot() AXES-LEVEL• **Definition**

Plots points at (x, y) , with optional color (hue), marker shape (style), and size (size) mapped to other columns. For classification this is how you visually check whether two features already separate the classes — which hints at whether a linear boundary (logistic regression, linear SVM) will work, or whether you need something non-linear (kernel SVM, trees, k-NN).

```
sns.scatterplot(data=None, x=None, y=None, hue=None, style=None,
                size=None, palette=None, alpha=1.0, ax=None)
```

PARAMETER	WHAT IT CONTROLS
<code>hue</code>	color by class label — the most common use in this unit
<code>style</code>	different marker shapes per category (useful alongside hue for colorblind-safe or print-friendly plots)
<code>size</code>	maps a numeric column to point size (e.g. a probability)
<code>palette</code>	color palette name or explicit mapping, e.g. "deep"

- **Every case**

- **Two features colored by class:** `hue="target"`
- **Add marker shape too:** `style="target"` together with `hue="target"` for redundant, accessible encoding
- **Show model confidence:** `size="predicted_proba"` — larger points are predictions the model is more confident about
- **Overlay a decision boundary:** scatter the true points, then draw a filled contour of model predictions underneath with Matplotlib's `contourf`

- **Worked example**

```
# Unit 4: are two features linearly separable?
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
idf = iris.frame
idf["species"] = idf["target"].map(dict(enumerate(iris.target_names)))

plt.figure(figsize=(6, 5))
sns.scatterplot(data=idf, x="petal length (cm)", y="petal width (cm)",
                hue="species", style="species", s=70, palette="deep")
plt.title("Petal measurements by species")
plt.show()
```

Setosa separates almost perfectly on these two features alone — a strong hint that even a simple linear model (logistic regression, linear SVM) will do well distinguishing it from the other two.

`sns.pairplot()` FIGURE-LEVEL

- **Definition**

Builds a full grid of pairwise scatterplots for every combination of numeric columns, with histograms (or KDEs) on the diagonal. It is the fastest way to scan *all* feature pairs for class separability at once, instead of calling `scatterplot` repeatedly.

```
sns.pairplot(data, hue=None, vars=None, diag_kind='auto',
             corner=False, palette=None, height=2.2)
```

PARAMETER	WHAT IT CONTROLS
<code>hue</code>	color every panel by class label
<code>vars</code>	restrict to a subset of columns (full grid gets large fast)
<code>diag_kind</code>	'hist' or 'kde' for the diagonal

- **Every case**

- **All features, colored by class:** `sns.pairplot(idf, hue="species")`
- **Restricted to a few features:** `vars=[...]` when the dataset has many columns (e.g. before k-NN, where the "curse of dimensionality" makes this scan more important)
- **Corner-only grid:** `corner=True` for a cleaner, non-redundant layout

- **Worked example**

```
sns.pairplot(idf.drop(columns="target"), hue="species",
             palette="deep", corner=True, diag_kind="kde")
plt.show()
```

This single figure tells you, before training anything, which feature pairs a decision tree would likely split on first, and whether Naive Bayes' assumption of roughly Gaussian, class-conditional features looks reasonable from the diagonal KDEs.

`sns.heatmap()` for a confusion matrix EVALUATION

- **Definition**

Same function as in Unit 2, applied to a different matrix: scikit-learn's `confusion_matrix` output. Rows are true classes, columns are predicted classes, and the diagonal is what the classifier got right — the visual basis for accuracy, precision, recall, and F1.

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)
```

- **Every case**

- **Raw counts:** `fmt='d'`, integer annotations
- **Normalized (row-wise) percentages:** pass `confusion_matrix(y_true, y_pred, normalize='true')` and use `fmt='.1%'` or `'.2f'` — shows recall per class directly
- **Binary classification, 2×2** — label cells TP / FP / FN / TN explicitly with `annot` as a custom string array

- **Worked example**

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

X, y = idf.drop(columns=["target", "species"], idf["target"])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

clf = LogisticRegression(max_iter=1000).fit(X_train, y_train)
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.title("Logistic regression – confusion matrix")
plt.show()

```

Boxplots and violin plots (from Unit 2's boxplot, plus `sns.violinplot()` which takes identical arguments) are also frequently reused here: `sns.violinplot(data=idf, x="species", y="petal length (cm)")` shows the full distribution shape per class, useful for judging Naive Bayes' Gaussian assumption per class.

EXERCISE – UNIT 4

Classification with seaborn

1. Load `load_wine`. Use `pairplot` with `hue="target"` restricted to 4 features (`vars=[...]`) to pick the two features that separate classes best.
2. Use `scatterplot` on just those two features, with both `hue` and `style` set to the class, sized by the value of a third feature.
3. Train a k-NN classifier (`k=5`) and a Decision Tree on the same data. Plot both confusion matrices side by side with `heatmap` and compare which classes get confused.
4. Re-plot one confusion matrix normalized by row (`normalize='true'`) and explain, in one sentence, how that changes the metric you're reading off the diagonal.

► [Show a starting-point solution](#)

UNIT 5

Clustering — no labels, so the plot is the evidence

Approaches to clustering, distance metrics, K-means, hierarchical clustering. Without ground-truth labels, a visualization is often the only way to judge whether clusters are sensible — and for hierarchical clustering specifically, seaborn has a function built for exactly this: the dendrogram-plus-heatmap combination.

`sns.scatterplot()` for cluster labels K-MEANS

• Definition

Identical function to Unit 4's version — the only difference is that hue now maps to the *cluster assignment* produced by an algorithm like K-means, not a true class label. Because there is no ground truth, this plot is often the main way to sanity-check k.

• Every case

- **Two raw features, colored by cluster:** `hue="cluster"`
- **Cluster centers overlaid:** a second, larger-marker scatterplot for `kmeans.cluster_centers_`, plotted on the same axes
- **Comparing values of k side-by-side:** one subplot per candidate k to visually pick an elbow

• Worked example

```
# Unit 5: K-means, visualized
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(wdf.drop(columns="target"))
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
wdf["cluster"] = km.labels_

plt.figure(figsize=(6, 5))
sns.scatterplot(data=wdf, x="flavanoids", y="color_intensity",
                hue="cluster", palette="deep", s=60)
plt.title("K-means clusters (k=3) on wine data")
plt.show()
```

Because K-means was fit on *all* scaled features but we're only viewing two, clusters may look imperfectly separated here even when they are well separated in the full feature space — a reminder that a 2-D scatter is a projection, not the whole picture.

`sns.clustermap()` HIERARCHICAL CLUSTERING

• Definition

Draws a heatmap where rows and/or columns are reordered according to a hierarchical clustering (agglomerative, using a distance metric and linkage method), with the resulting dendrograms drawn alongside. This is the one seaborn function that directly visualizes the hierarchical clustering algorithm itself, rather than just its output.

```
sns.clustermap(data, metric='euclidean', method='average',
               z_score=None, standard_scale=None, cmap='vlag',
               row_cluster=True, col_cluster=True, figsize=(10, 10))
```

PARAMETER	WHAT IT CONTROLS
<code>metric</code>	distance metric between rows, e.g. <code>'euclidean'</code> , <code>'cosine'</code> , <code>'cityblock'</code>
<code>method</code>	linkage method: <code>'average'</code> , <code>'single'</code> , <code>'complete'</code> , <code>'ward'</code>
<code>z_score</code>	standardize rows (0) or columns (1) before clustering, so scale doesn't dominate distance
<code>row_cluster</code> , <code>col_cluster</code>	toggle clustering off on either axis

• Every case

- **Cluster samples only** (rows), keep original feature order: `col_cluster=False`
- **Cluster both samples and features**: default behavior — reveals feature groups too
- **Different linkage methods**: compare `method='single'` (chaining) vs `method='ward'` (compact, even-sized clusters) on the same data
- **Standardized features first**: `z_score=1` so no single large-scale feature dominates the distance metric
- **Color rows by true/cluster label**: pass `row_colors=` a Series mapping each sample to a color, to visually check hierarchical clusters against known labels

• Worked example

```
# Unit 5: hierarchical clustering, visualized directly
sample = wdf.drop(columns=["target", "cluster"]).sample(40, random_state=0)

palette = dict(zip(wdf["target"].unique(), sns.color_palette("deep", 3)))
row_colors = wdf.loc[sample.index, "target"].map(palette)

sns.clustermap(sample, method="ward", metric="euclidean",
               z_score=1, cmap="vlag", row_colors=row_colors,
               figsize=(9, 9))

plt.show()
```

The colored strip next to the dendrogram shows the *true* wine class for each row. If the dendrogram's branches roughly group same-colored rows together, hierarchical clustering has recovered structure that lines up with the real classes — without ever being told what they were.

EXERCISE — UNIT 5

Clustering with seaborn

1. Run K-means with $k = 2, 3, 4, 5$ on the scaled wine features. For each, scatterplot two features colored by cluster in a 2×2 grid of subplots. Pick the k that looks most visually sensible and explain why.
2. Build a `clustermap` on a 40-row sample of the (standardized) wine data using `method= 'ward '`, then rebuild it with `method= 'single '`. Describe one visible difference between the two dendrograms.
3. Add `row_colors` mapped to the true wine class and comment on whether the hierarchical clusters recover the true classes better or worse than your chosen K-means k did in question 1.

► [Show a starting-point solution](#)

REFERENCE

One-page cheat sheet

UNIT	QUESTION YOU'RE ASKING	FUNCTION
2 — Preprocessing	What shape is this feature?	<code>sns.histplot()</code>
2 — Preprocessing	Which features have outliers?	<code>sns.boxplot()</code>
2 — Preprocessing	Which features are redundant / what drives each PC?	<code>sns.heatmap()</code>
3 — Regression	Does a line fit one relationship well?	<code>sns.regplot()</code>
3 — Regression	Does the fit differ across groups / multiple variables?	<code>sns.lmplot()</code>
3 — Regression	Are the residuals well-behaved?	<code>sns.residplot()</code>
4 — Classification	Do two features separate the classes?	<code>sns.scatterplot()</code>
4 — Classification	Which feature pair separates classes best, overall?	<code>sns.pairplot()</code>
4 — Classification	Where does the classifier get confused?	<code>sns.heatmap()</code> (confusion matrix)
5 — Clustering	Do K-means clusters look sensible?	<code>sns.scatterplot()</code>
5 — Clustering	How does hierarchical clustering group the data?	<code>sns.clustermap()</code>